UCS645

PARALLEL & DISTRIBUTED COMPUTING

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

ASSIGNMENT 4:

Submitted by: Sujal Mallick (102303266)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Ring Communication

1. Objective

To implement a parallel MPI program that demonstrates inter-process communication using a ring topology. The experiment evaluates message passing, process coordination, and correctness of data transfer across processes.

2. Problem Description

In a ring communication pattern, each process sends a message to the next process in sequence.

- Process 0 starts with an initial value of 100
- Each process adds its rank to the received value
- The message is passed along the ring
- The last process sends the final value back to Process 0

The goal is to ensure correct cyclic communication and data propagation across all processes.

3. Methodology

1. Initialized MPI environment using MPI_Init()
2. Obtained process rank and total number of processes
3. Process 0 initialized the value to 100 and sent it to Process 1
4. Each process:
   Received value from previous process
   Added its rank
   Sent updated value to next process
5. Used modulo operation: to maintain ring topology
6. Final value returned to Process 0

4. Performance Results

Processes	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000098	1.00	100.0%
2	0.000112	0.88	43.9%
4	0.000167	0.59	14.7%
8	0.001823	0.05	0.7%

5. Performance Discussion

Ring communication is latency-dominated — adding more processes increases hops in the ring, so execution time goes up instead of down, which is why speedup drops below 1.0 and efficiency collapses at 8 processes. This is expected behavior for this pattern.

6. Conclusion

The program successfully demonstrates ring communication using MPI. Correct data propagation across processes confirms proper implementation of point-to-point communication

Parallel Array Sum

1. Objective

To compute the sum of an array using parallel processing in MPI and evaluate collective communication using MPI_Scatter and MPI_Reduce.

2. Problem Description

An array of size 100 is initialized with values from 1 to 100. The array is divided among processes, and each process computes a local sum. The global sum is obtained using reduction.

3. Methodology

1. Process 0 initialized array (1–100)
2. Distributed data using MPI_Scatter()
3. Each process computed local sum
4. Combined results using MPI_Reduce()
5. Computed average value

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000028	1.00	100.0%
2	0.000019	1.47	73.7%
4	0.000016	1.75	43.8%
8	0.000021	1.33	16.6%

5. Performance Discussion

1. Speedup peaks at 4 processes (1.75×) — beyond that, MPI communication overhead outweighs computation gains.
2. Efficiency drops from 73.7% to 16.6% as processes increase — array is too small (100 elements) to justify 8 processes.
3. 8 processes slower than 4 — problem is communication-bound; coordinating 8 processes costs more than the parallel benefit.
4. Sequential bottleneck limits scaling — array initialization at Process 0 cannot be parallelized, capping max speedup.

6. Conclusion

MPI_Scatter and MPI_Reduce efficiently distribute and combine data. The program demonstrates effective parallel aggregation with correct results.

Maximum and Minimum

1. Objective
To compute global maximum and minimum values across processes using MPI reduction operations.

2. Problem Description
Each process generates 10 random numbers (0–1000). Each process computes local maximum and minimum. Global values are determined using reduction operations.

3. Methodology

1. Each process generated random numbers
2. Computed local max and min
3. Used:
    - MPI_MAXLOC for global max
    - MPI_MINLOC for global min
4. Stored value + rank using structure

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000022	1.00	100.0%
2	0.000015	1.47	73.3%
4	0.000011	2.00	50.0%
8	0.000018	1.22	15.3%

5. Performance Discussion

1. **Speedup peaks at 4 processes (2.00×)** — each process handles only 10 random numbers independently, so parallel reduction via MPI_MAXLOC/MPI_MINLOC is effective up to 4 processes.
2. **Efficiency drops from 73.3% to 15.3%** — small workload (10 numbers per process) means communication cost of two MPI_Reduce calls dominates at higher process counts.
3. **8 processes slower than 4** — two sequential MPI_Reduce operations (one for max, one for min) double the communication overhead, making 8 processes inefficient for such a tiny dataset.

6. Conclusion
The program demonstrates MPI reduction with location tracking. Efficient computation of global extrema across distributed processes was achieved

Parallel Dot Product

1. Objective
To compute the dot product of two vectors using MPI-based parallel processing.

2. Problem Description
Two vectors:

- A = [1,2,3,4,5,6,7,8]
- B = [8,7,6,5,4,3,2,1]

Vectors are distributed across processes. Each process computes partial dot product, and results are combined.

3. Methodology

1. Distributed vectors using MPI_Scatter()
2. Each process computed partial dot product
3. Used MPI_Reduce() to combine results
4. Final result computed at Process 0

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000159	1.00	100.0%
2	0.000098	1.62	81.1%
4	0.000071	2.24	56.0%
8	0.000113	1.41	17.6%

5. Performance Discussion

1. Speedup peaks at 4 processes (2.24×) — two MPI_Scatter calls efficiently split both vectors, and MPI_Reduce cleanly aggregates partial dot products with minimal overhead at 4 processes.
2. Efficiency drops from 81.1% to 17.6% — vector size is only 8 elements, so each process computes just 1–2 multiplications while paying full MPI communication cost.
3. 8 processes slower than 4 — with only 8 elements, each process gets just 1 element at np=8, making communication overhead completely dominate over computation.

6. Conclusion
MPI_Scatter and MPI_Reduce efficiently distribute and combine data. The program demonstrates effective parallel aggregation with correct results.